

Double-click (or enter) to edit

```
! pip install langchain
!pip!pip install --quiet requests beautifulsoup4
```

```
Requirement already satisfied: langchain in /usr/local/lib/python3.11/dist-packages (0.3.24)
Requirement already satisfied: langchain-core<1.0.0,>=0.3.55 in /usr/local/lib/python3.11/dist-packages (from langchain) (0.3.56)
Requirement already satisfied: langchain-text-splitters<1.0.0,>=0.3.8 in /usr/local/lib/python3.11/dist-packages (from langchain) (0.3.8)
Requirement already satisfied: langsmith<0.4,>=0.1.17 in /usr/local/lib/python3.11/dist-packages (from langchain) (0.3.38)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.11/dist-packages (from langchain) (2.11.3)
Requirement already satisfied: SQLAlchemy<3,>=1.4 in /usr/local/lib/python3.11/dist-packages (from langchain) (2.0.40)
Requirement already satisfied: requests<3,>=2 in /usr/local/lib/python3.11/dist-packages (from langchain) (2.32.3)
Requirement already satisfied: PyYAML<=5.3 in /usr/local/lib/python3.11/dist-packages (from langchain) (6.0.2)
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.55->langchain) (8.5.0)
Requirement already satisfied: jsonpatch<2.0,>=1.33 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.55->langchain) (1.33)
Requirement already satisfied: packaging<25,>=23.2 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.55->langchain) (24.0)
Requirement already satisfied: typing-extensions>=4.7 in /usr/local/lib/python3.11/dist-packages (from langchain-core<1.0.0,>=0.3.55->langchain) (4.11.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.17->langchain) (0.27.0)
Requirement already satisfied: orjson<4.0.0,>=3.9.14 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.17->langchain) (3.10.7)
Requirement already satisfied: requests-toolbelt<2.0.0,>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.17->langchain) (1.0.0)
Requirement already satisfied: zstandard<0.24.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from langsmith<0.4,>=0.1.17->langchain) (0.23.0)
Requirement already satisfied: annotated-types<=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (0.6.0)
Requirement already satisfied: pydantic-core==2.33.1 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (2.33.1)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<3.0.0,>=2.7.4->langchain) (0.4.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langchain) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langchain) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langchain) (2.4.0)
Requirement already satisfied: certifi<=2025.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2->langchain) (2025.4.17)
Requirement already satisfied: greenlet<=1 in /usr/local/lib/python3.11/dist-packages (from SQLAlchemy<3,>=1.4->langchain) (3.2.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->langsmith<0.4,>=0.1.17->langchain) (4.3.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx<1,>=0.23.0->langsmith<0.4,>=0.1.17->langchain) (1.0.5)
Requirement already satisfied: h11<=0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->langsmith<0.4,>=0.1.17->langchain) (0.14.0)
Requirement already satisfied: jsonpointer<=1.9 in /usr/local/lib/python3.11/dist-packages (from jsonpatch<2.0,>=1.33->langchain-core<1.0.0,>=0.3.55->langchain) (2.3.0)
Requirement already satisfied: sniffio<=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio->httpx<1,>=0.23.0->langsmith<0.4,>=0.1.17->langchain) (1.3.1)
/bin/bash: line 1: pip!pip: command not found
```

Imports & Scraper Function

```
# Cell 2 - Imports & updated scraper
import requests, os
from bs4 import BeautifulSoup
from urllib.parse import urlparse

# <<< ADD this line >>>
DRIVE_DIR = "/content/drive/MyDrive/Colab Notebooks/News_text"

def scrape_and_save(url, output_dir=DRIVE_DIR):
    """
    Fetch <p>-tag text from a URL and save it to
    /content/drive/MyDrive/Colab Notebooks/News_text/<domain>.txt
    """
    headers = {'User-Agent': 'Mozilla/5.0'}
    resp = requests.get(url, headers=headers)
    resp.raise_for_status()

    soup = BeautifulSoup(resp.text, 'html.parser')
    paragraphs = soup.find_all('p')
    text_content = '\n\n'.join(p.get_text(strip=True) for p in paragraphs)

    domain = urlparse(url).netloc.replace('.', '_')
    filename = f"{domain}.txt"
    os.makedirs(output_dir, exist_ok=True) # ensure Drive folder exists
    file_path = os.path.join(output_dir, filename)

    with open(file_path, 'w', encoding='utf-8') as f:
        f.write(text_content)

    print(f"✓ Saved {file_path}")
```

✓ Prompt for URLs

Cell 3 – Three text boxes for URL input

```
import ipywidgets as widgets
from IPython.display import display

url_box1 = widgets.Text(
    description='URL 1:',
    placeholder='https://www.cnn.com/business'
)
url_box2 = widgets.Text(
    description='URL 2:',
    placeholder='https://www.cnn.com/'
)
url_box3 = widgets.Text(
    description='URL 3:',
    placeholder='https://finance.yahoo.com/'
)

display(url_box1, url_box2, url_box3)
```

After typing URLs, simply run the next cell.

The next cell will read `url\_box1.value`, `url\_box2.value`, `url\_box3.value`.



URL 1:

URL 2:

URL 3:

NEW Cell 4 – Collect the URLs from the widgets

```
urls = [u.strip() for u in (url_box1.value, url_box2.value, url_box3.value) if u.strip()][:3]

if not urls:
    raise ValueError("Please enter at least one valid URL in the boxes above and rerun this cell.")

for url in urls:
    try:
        scrape_and_save(url)
    except Exception as e:
        print(f"✗ Failed to scrape {url}: {e}")
```



- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/www_cnn_com.txt
- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/www_cnn_com.txt
- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/finance_yahoo_com.txt

✓ Run the Scraper

```
for url in urls:
    try:
        scrape_and_save(url)
    except Exception as e:
        print(f"✗ Failed to scrape {url}: {e}")
```



- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/www_cnn_com.txt
- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/www_cnn_com.txt
- ✓ Saved /content/drive/MyDrive/Colab Notebooks/News_text/finance_yahoo_com.txt

✓ List Saved Files

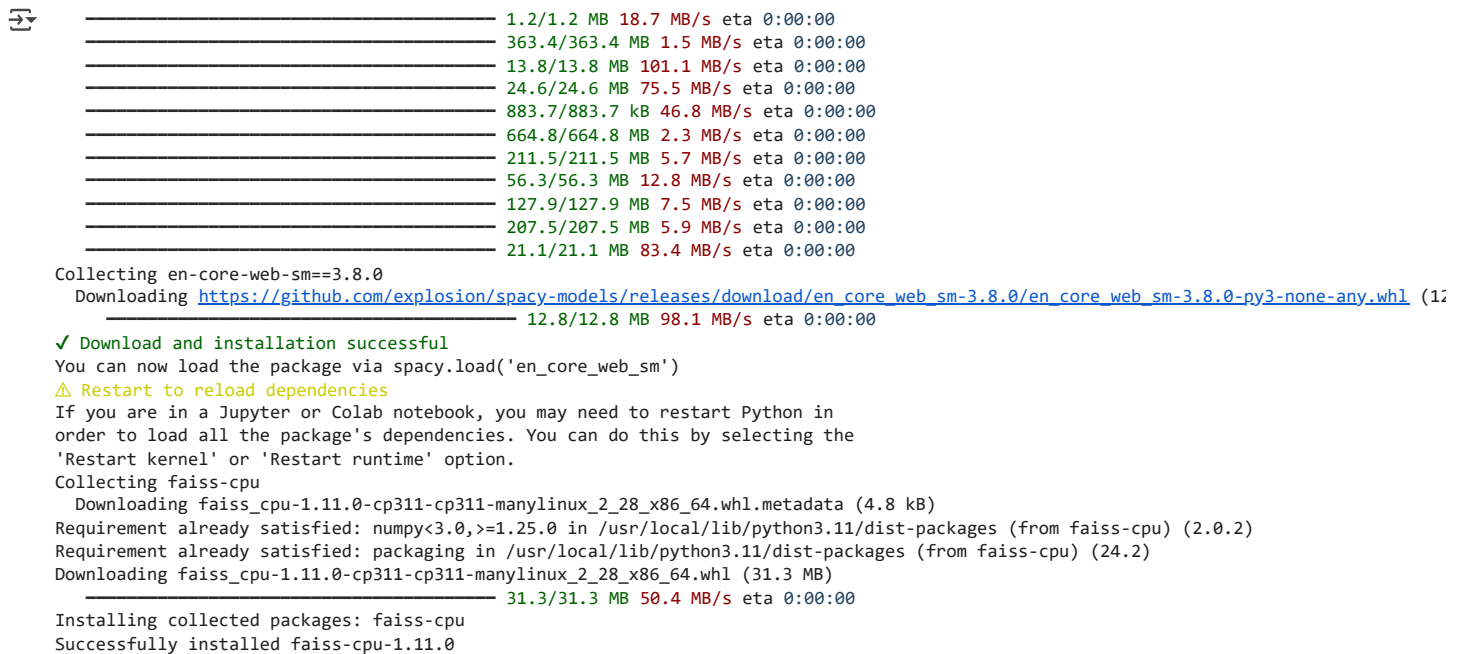
```
!ls -1 "/content/drive/MyDrive/Colab Notebooks/News_text"
```



```
finance_yahoo_com.txt
semantic_chunks
www_bloomberg_com.txt
www_cnn_com.txt
www_cnn_com.txt
```

✓ Semantic chunking utilities

```
!pip install --quiet sentence-transformers spacy tiktoken
!python -m spacy download en_core_web_sm
!pip install faiss-cpu
```



```

1.2/1.2 MB 18.7 MB/s eta 0:00:00
363.4/363.4 MB 1.5 MB/s eta 0:00:00
13.8/13.8 MB 101.1 MB/s eta 0:00:00
24.6/24.6 MB 75.5 MB/s eta 0:00:00
883.7/883.7 kB 46.8 MB/s eta 0:00:00
664.8/664.8 MB 2.3 MB/s eta 0:00:00
211.5/211.5 MB 5.7 MB/s eta 0:00:00
56.3/56.3 MB 12.8 MB/s eta 0:00:00
127.9/127.9 MB 7.5 MB/s eta 0:00:00
207.5/207.5 MB 5.9 MB/s eta 0:00:00
21.1/21.1 MB 83.4 MB/s eta 0:00:00

Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8/12.8 MB 98.1 MB/s eta 0:00:00)
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.
Collecting faiss-cpu
  Downloading faiss_cpu-1.11.0-cp311-cp311-manylinux_2_28_x86_64.whl.metadata (4.8 kB)
Requirement already satisfied: numpy<3.0,>=1.25.0 in /usr/local/lib/python3.11/dist-packages (from faiss-cpu) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from faiss-cpu) (24.2)
  Downloading faiss_cpu-1.11.0-cp311-cp311-manylinux_2_28_x86_64.whl (31.3 MB)
    31.3/31.3 MB 50.4 MB/s eta 0:00:00
Installing collected packages: faiss-cpu
Successfully installed faiss-cpu-1.11.0

```

```

from pathlib import Path
import spacy
import numpy as np, faiss, tiktoken, os, json, math
from sentence_transformers import SentenceTransformer
from tqdm.auto import tqdm

RAW_DIR = Path("/content/drive/MyDrive/Colab Notebooks/News_text")
CLEAN_DIR = RAW_DIR / "semantic_chunks"
CLEAN_DIR.mkdir(exist_ok=True, parents=True)

NLP = spacy.load("en_core_web_sm")
EMB_MODEL = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
ENCODER = tiktoken.get_encoding("cl100k_base") # counts tokens like OpenAI

MAX_TOKENS = 256 # hard cap per chunk
SIM_THRESH = 0.72 # cosine similarity threshold between sentences

def token_len(text:str) -> int:
    return len(ENCODER.encode(text))

def cosine(a,b):
    # a, b are 1-D numpy arrays
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))

def semantic_chunk(text:str,
                  max_tokens:int = MAX_TOKENS,
                  sim_thresh:float = SIM_THRESH):
    """Yield semantically coherent chunks from a full article."""
    doc = NLP(text)
    sentences = [sent.text.strip() for sent in doc.sents if sent.text.strip()]
    if not sentences:
        return []

    sent_embeddings = EMB_MODEL.encode(sentences, convert_to_numpy=True,
                                       normalize_embeddings=True)

    chunks, cur_chunk, cur_embs = [], [], []
    cur_tokens = 0

    for sent, emb in zip(sentences, sent_embeddings):
        sent_tokens = token_len(sent)

```

```

# start new chunk if sentence alone is huge
if sent_tokens >= max_tokens:
    if cur_chunk:
        chunks.append(" ".join(cur_chunk))
        cur_chunk, cur_embs, cur_tokens = [], [], 0
    chunks.append(sent)
    continue

if not cur_chunk:
    # new chunk
    cur_chunk.append(sent)
    cur_embs.append(emb)
    cur_tokens += sent_tokens
    continue

# similarity w.r.t. mean embedding of current chunk
centroid = np.mean(cur_embs, axis=0)
sim = cosine(centroid, emb)

token_ok = (cur_tokens + sent_tokens) <= max_tokens
sem_ok = sim >= sim_thresh

if token_ok and sem_ok:
    cur_chunk.append(sent)
    cur_embs.append(emb)
    cur_tokens += sent_tokens
else:
    # flush current chunk
    chunks.append(" ".join(cur_chunk))
    cur_chunk, cur_embs, cur_tokens = [sent], [emb], sent_tokens

if cur_chunk:
    chunks.append(" ".join(cur_chunk))
return chunks

```

 /usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 10.8kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 5.27kB/s]
README.md: 100% 10.5k/10.5k [00:00<00:00, 677kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 4.02kB/s]
config.json: 100% 612/612 [00:00<00:00, 34.1kB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, please install the 'hf_xet' package.
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 193MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 17.6kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 4.77MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 20.6MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 11.5kB/s]
config.json: 100% 190/190 [00:00<00:00, 10.7kB/s]

```

✓ Process every scraped article

```

for txt_file in tqdm(list(RAW_DIR.glob("*.txt"))):
    text = txt_file.read_text(encoding="utf-8").strip()
    if not text:
        continue
    chunks = semantic_chunk(text)

    stem = txt_file.stem

```

```

for i, chunk in enumerate(chunks):
    out = CLEAN_DIR / f"{stem}_{i:03d}.txt"
    out.write_text(chunk, encoding="utf-8")
print(f"🟢 Finished. {len(list(CLEAN_DIR.glob('*.txt')))} chunks saved to {CLEAN_DIR}")

```



100%

4/4 [00:07<00:00, 1.95s/it]

🟢 Finished. 39 chunks saved to /content/drive/MyDrive/Colab Notebooks/News_text/semantic_chunks

✓ Embed & Index

```

# Cell D - Embed chunks and build FAISS
!pip install -q sentence_transformers faiss-cpu

from sentence_transformers import SentenceTransformer
import faiss, numpy as np, json, os
from pathlib import Path
from tqdm.auto import tqdm

CHUNK_DIR = Path("/content/drive/MyDrive/Colab Notebooks/News_text/semantic_chunks")
INDEX_DIR = Path("/content/drive/MyDrive/Colab Notebooks/News_index")
INDEX_DIR.mkdir(exist_ok=True, parents=True)

model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
dim = model.get_sentence_embedding_dimension()
index = faiss.IndexFlatIP(dim) # cosine = dot if vectors are normed
meta = {} # vector_id → {path, url?}

vec_id = 0
for f in tqdm(sorted(CHUNK_DIR.glob("*.txt"))):
    text = f.read_text(encoding="utf-8")
    emb = model.encode([text], normalize_embeddings=True)
    index.add(emb)
    meta[vec_id] = {"path": str(f), "preview": text[:160] + "..."}
    vec_id += 1

faiss.write_index(index, str(INDEX_DIR / "news.index"))
json.dump(meta, open(INDEX_DIR / "news_meta.json", "w"))
print(f"📁 Indexed {vec_id} chunks → {INDEX_DIR}")

```



100%

39/39 [00:02<00:00, 19.50it/s]

📁 Indexed 39 chunks → /content/drive/MyDrive/Colab Notebooks/News_index

✓ Groq-Cloud RAG Function

```

from google.colab import userdata
userdata.get('GROQ_API_KEY')

```



'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'

```

# ▶ Run this cell AFTER you have:
# 1. Mounted Drive (drive.mount('/content/drive'))
# 2. Built / saved news.index and news_meta.json
# 3. Added GROQ_API_KEY in Colab "Secrets"

```

```

from pathlib import Path
from google.colab import userdata
from openai import OpenAI
import json, faiss, numpy as np, tiktoken
from sentence_transformers import SentenceTransformer

# — 1. Groq client —————
GROQ_KEY = userdata.get("GROQ_API_KEY")
assert GROQ_KEY, "Add GROQ_API_KEY in Secrets (🔑 icon at left)."

client = OpenAI(
    api_key = GROQ_KEY,
    base_url = "https://api.groq.com/openai/v1",

```

```

)
MODEL_NAME = "llama3-70b-8192"

# — 2. Load FAISS index + metadata —————
INDEX_DIR = Path("/content/drive/MyDrive/Colab Notebooks/News_index")
index = faiss.read_index(str(INDEX_DIR / "news.index"))
meta = json.load(open(INDEX_DIR / "news_meta.json")) # meta is a LIST

# — 3. Embedding & tokenizer —————
embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
enc = tiktoken.get_encoding("cl100k_base")
tok_len = lambda txt: len(enc.encode(txt))

# — 4. Retrieval helper —————
def retrieve(query, k=4):
    vec_q, _ = embedder.encode([query], normalize_embeddings=True, convert_to_numpy=True, show_progress_bar=False)
    _, ids = index.search(vec_q, k)
    return [meta[int(i)] for i in ids[0]]

# — 5. RAG + Groq generation —————
def rag_groq(question, k=4, max_ctx_tokens=1500, temperature=0.1):
    context_chunks = []
    total_tokens = 0
    for m in retrieve(question, k):
        chunk = Path(m["path"]).read_text(encoding="utf-8")
        if total_tokens + tok_len(chunk) > max_ctx_tokens:
            break
        context_chunks.append(chunk)
        total_tokens += tok_len(chunk)

    context = "\n\n--\n\n".join(context_chunks)
    prompt = f"""You are a finance-news assistant.
Answer using ONLY the context below; cite sources by domain.
If unsure, say you don't know.

Context:
{context}

Question: {question}
Answer: """

    resp = client.chat.completions.create(
        model = MODEL_NAME,
        messages = [{"role": "user", "content": prompt}],
        temperature = temperature,
        max_tokens = 300,
    )
    return resp.choices[0].message.content.strip()

```

```

def retrieve(query, k=4):
    # embed the query
    qvec = embedder.encode([query], normalize_embeddings=True)
    # search FAISS
    _, I = index.search(np.array(qvec), k)

    hits = []
    for i in I[0]:
        key = str(i) # meta keys are strings
        if key in meta: # skip if missing
            hits.append(meta[key])
    return hits

```

▼ USAGE

```
print(rag_groq("What are the latest news on Recession ?"))
```

 I don't know. The provided context does not provide any information about the latest news on recession. It mentions Trump's tariff reven

Gradio Chat

```
!pip install -q gradio

import gradio as gr

def chat_fn(q):
    try:
        return rag_groq(q)
    except Exception as e:
        return f"Error: {e}"

demo = gr.Interface(chat_fn,
                    gr.Textbox(lines=2, label="Ask the news bot"),
                    gr.Markdown(label="Answer"),
                    title="Finance-News RAG (Llama-3 on Groq)")
demo.launch(share=True)
```

54.1/54.1 MB13.1 MB/s eta 0:00:00

322.9/322.9 kB20.9 MB/s eta 0:00:00

95.2/95.2 kB7.8 MB/s eta 0:00:00

11.5/11.5 MB100.6 MB/s eta 0:00:00

72.0/72.0 kB6.2 MB/s eta 0:00:00

62.5/62.5 kB5.2 MB/s eta 0:00:00

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: <https://dfb571a004ca32102d.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run ``gradio deploy`` from the terminal in the working dir

Finance-News RAG (Llama-3 on Groq)

Ask the news bot

what is the latest news on US economy

Clear

Submit

According to Bloomberg (bloomberg.com), Trump's April tariff revenue topped \$17 billion, which is his biggest haul yet. However, the article also mentions that policy and economic shifts are on the horizon, indicating that there may be changes coming to the US economy.

Flag